certora

Security Assessment

Final Report



Spectra Core

July 2026

Prepared for Spectra

certora

Table of Contents

Project Summary **4**
- Project Scope 4
- Project Overview 4
- Protocol Overview 6

Assessment Methodology **7**

Threat and Security Overview **8**
- 1. System-level model and security-relevant elements 8
- 2. Expected invariants 10
- 3. Trust boundaries and security assumptions 11
- 4. Attack vectors considered 12
- 5. Components/flows manually reviewed, invariants checked, behaviors tested 13
- 6. Test coverage - gaps and caveats 14
- Findings Summary 15
- Severity Matrix 15

Detailed Findings **16**
- High Severity Issues 20
  - H-01: Blend b_rate is not guaranteed to be monotonic and can corrupt accounting 20
- Medium Severity Issues 22
  - M-01: Fail-safe rate uses input-token decimals for output-token floor 22
  - M-02: LOE: missing authorize_pull makes Underlying/VaultShare-funded fills always revert in production 24
  - M-03: Reported output of external aggregator is trusted 26
  - M-04: Fees are changeable on already-accrued YT yield 28
  - M-05: Both Router bridge-out commands revert on every successful call due to mis-declared return type 29
  - M-06: Bridging a PT with underfunded gas commits a local PT lock or wrapped-PT burn while the cross-chain delivery never happens 31
- Low Severity Issues 33
  - L-01: Admin transfer requires manual rollover of operational roles 33
  - L-02: Zero YT price lets IbtForYt orders register but revert during fill 34
  - L-03: Permissionless deploy_pt poisons registered PT list 36
  - L-04: Rewards-proxy can arbitrarily "call-as-self", granting too much power to the harvester role 38
  - L-05: LimitOrderEngine pulls taker_input_token before validating it, stranding unsupported non-swept tokens 41
  - L-06: Zero-amount third-party redeem/withdraw lets any caller force another user's yield checkpoint 43
  - L-07: Router AssertMinBalance can miss under-delivery for pre-funded callers 46
  - L-08: pricing_data returns a stale stored IBT rate where Solidity recomputes it live 48
  - L-09: RedeemPtForIbt drops Solidity's pre-maturity YT-balance clamp 50
  - L-10: Unauthenticated initialize is front-runnable on every contract 51

L-11: Per-user tokenization-fee reductions are keyed to the Router/engine's own address, not the user.. 52
Informational Severity Issues 54
I-01: Partial fills can settle with zero maker proceeds 54
I-02: Payout-side tokenisation creates an unused YT approval 56
I-03: PT deployment salt IBT inclusion should be considered 58
I-04: PT initialization can accept a zero initial IBT rate 59
I-05: Behalf-of redeem requires an undocumented YT allowance 61
I-06: Wrapper max_ function quotes are liquidity-blind 62
I-07: Blend-backed LOE fillability is route-dependent under current Soroban memory limits 63
I-08: Router bridge commands should reject negative signed amounts before calling the bridge 64
I-09: LOE fill interface docs incorrectly require taker approval to the engine 66
I-10: set_secs_per_ledger uses a raw assert instead of a structured contract error 68
I-11: Zero-amount YT transfer permissionlessly triggers before_yt_transfer on any holder 69
I-12: Registry template and upgrade hashes are insta-changeable 70
I-13: Internal token approvals use inconsistent expiry ledgers (seq+1 vs seq) 71
I-14: Long-metadata tokens can get blocked out of deployment 73
Disclaimer 74
About Certora 74

certora

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Spectra Core	https://github.com/perspectivefi/spectra-core-stellar/tree/684288baa095ad182c0fc0b8a2ce3a85871f055c https://github.com/perspectivefi/spectra-core-stellar/tree/b1f0641e790ae36a474ebed4b360b866caebc7bb	684288b (initial commit) b1f0641 (fix commit)	Stellar

Project Overview

This document describes the security review of **Spectra Core**. The work was undertaken from **June 29th, 2026** to **July 20th, 2026**.

The following contract list is included in our scope:

contracts/blend-fungible-vault-wrapper/src/errors.rs
contracts/blend-fungible-vault-wrapper/src/lib.rs
contracts/blend-fungible-vault-wrapper/src/storage.rs
contracts/factory/src/errors.rs
contracts/factory/src/events.rs
contracts/factory/src/lib.rs
contracts/factory/src/storage.rs
contracts/limit-order-engine/src/errors.rs
contracts/limit-order-engine/src/events.rs

contracts/limit-order-engine/src/lib.rs
contracts/limit-order-engine/src/pricing.rs
contracts/limit-order-engine/src/storage.rs
contracts/principal-token/src/errors.rs
contracts/principal-token/src/events.rs
contracts/principal-token/src/lib.rs
contracts/principal-token/src/storage.rs
contracts/registry/src/errors.rs
contracts/registry/src/events.rs
contracts/registry/src/lib.rs
contracts/registry/src/storage.rs
contracts/router/src/dispatcher.rs
contracts/router/src/errors.rs
contracts/router/src/events.rs
contracts/router/src/lib.rs
contracts/router/src/storage.rs
contracts/spectra-common/src/auth.rs
contracts/spectra-common/src/bridge_traits.rs
contracts/spectra-common/src/constants.rs
contracts/spectra-common/src/lib.rs
contracts/spectra-common/src/limit_order_engine.rs
contracts/spectra-common/src/naming.rs
contracts/spectra-common/src/principal_token.rs
contracts/spectra-common/src/registry.rs
contracts/spectra-common/src/router_types.rs
contracts/spectra-common/src/ttl.rs
contracts/spectra-common/src/yield_token.rs
contracts/spectra-fungible-vault-wrapper/src/errors.rs
contracts/spectra-fungible-vault-wrapper/src/events.rs
contracts/spectra-fungible-vault-wrapper/src/lib.rs
contracts/spectra-fungible-vault-wrapper/src/storage.rs
contracts/spectra-math/src/lib.rs
contracts/spectra-math/src/mul_div.rs
contracts/spectra-math/src/ray_math.rs

contracts/yield-token/src/errors.rs
contracts/yield-token/src/lib.rs
contracts/yield-token/src/storage.rs

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Spectra Core is a yield-tokenization protocol implemented on Stellar's Soroban smart-contract platform, ported from the Spectra V2 protocol originally written in Solidity. Its purpose is to separate the two economic components bundled inside any yield-bearing token — the principal and the future yield - so that each can be held, valued, and traded independently. A user who owns an interest-bearing token, such as a wrapper over a Blend lending-pool position, can deposit it into Spectra and receive back two distinct SEP-41 tokens: a Principal Token (PT), which represents a claim redeemable for the underlying asset at the market's maturity date, and a Yield Token (YT), which accrues the variable yield generated by the deposited position until that maturity. This split turns a single floating-return instrument into a fixed-income leg and a variable-income leg, enabling fixed-rate lending, yield speculation, and hedging strategies that are not possible with the raw yield-bearing token alone.

Conversions between the underlying asset and the IBT are delegated to a vault wrapper: an abstract wrapper base defines ERC-4626-style deposit, redeem, and withdraw semantics, while the concrete Blend adapter implements them against a Blend lending pool, caching the pool's bToken-to-underlying exchange rate for the duration of a ledger.

Surrounding this core are two coordination layers that make the tokens usable in practice. The `LimitOrderEngine` allows makers to register signed limit orders for PT and YT and lets takers fill them; settlement happens atomically by pulling both sides' tokens, converting them to IBT through the vault wrapper, pricing each order from an implied-rate discount curve and minting or redeeming PT/YT as needed to close the trade. The `Router` provides a multicall interface and the composition point for external integrations, dispatching swaps to the `StellarBroker` DEX aggregator and cross-chain transfers to a bridge stack that locks or burns PT on Stellar and delivers a wrapped representation to an EVM chain.

The protocol's security posture therefore depends not only on its internal accounting and rate math but also on a small number of external dependencies that sit at its boundaries. The Blend

lending pool serves as the backing vault and the source of the `b_rate` exchange index, and its behavior directly affects wrapper valuation. The `StellarBroker` aggregator is invoked for swap legs within Router multicalls, and the bridge stack handles cross-chain PT delivery. These external systems are integrated through contract-client traits and are outside the audited scope, so the correctness of the interfaces to them is part of the protocol's overall attack surface.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

certora

Threat and Security Overview

This section states the system-level security model of Spectra Core, the invariants and trust boundaries it relies on, the attack surface examined during the review, and the coverage and gaps of the accompanying test suite. It closes with the risk themes that dominate the protocol's residual exposure.

1. System-level model and security-relevant elements

Spectra Core is a Soroban/Stellar Rust port of the Spectra V2 yield-tokenization protocol. Its purpose is to split a yield-bearing token (an IBT — here a Blend `bToken` reached through a vault wrapper) into a Principal Token , redeemable for the underlying at maturity, and a Yield Token that accrues yield until maturity. Around that core sit a limit-order engine, a multicall router, and a permissionless factory/registry pair.

The security-relevant components are:

- `PrincipalToken` (PT) - Custodian of the IBT backing and the source of truth for rates, per-user yield checkpoints, tokenization/yield fees, and maturity rate freeze. Drives its paired YT. Also carries the rewards-proxy `invoke_contract` primitive.
- `YieldToken` (YT) - SEP-41 token; mint/burn gated to the paired PT; transfers settle both parties' yield via `before_yt_transfer`; maturity guards.
- `blend-fungible-vault-wrapper` - Adapts the external Blend lending pool: caches the pool `b_rate` per ledger timestamp, treats `1 share = 1 bToken`, and moves value via Blend `submit(SUPPLY/WITHDRAW)`. The `position ≥ supply` solvency invariant lives here.
- `spectra-fungible-vault-wrapper` - Abstract wrapper base (wrap/unwrap, ERC-4626-style views, rewards).
- `LimitOrderEngine` (LOE) - Matches signed PT/YT limit orders and settles them by minting/redeeming through the PT and wrapper in one atomic transaction. The protocol's heaviest transaction and its main value-flow chokepoint.

- `Router` - Composes user actions into an atomic `Vec<Command>` multicall with `Amount::Exact` / `Amount::Balance` sentinels; holds the authoritative peripheral address book; owns the nested-authorization pre-declarations for external broker/bridge calls.
- `Factory` + `Registry` - Permissionless PT/YT market deployment; fee parameters; deployable WASM hashes; peripheral address book.
- `spectra-math` - Fixed-point core: `mul_div` (I256 intermediates), 27-decimal `ray_math`, and `log_exp_math` for the order-pricing discount curve. All accounting bottoms out here.

Several Soroban execution-model elements are first-class to the threat model because they have no EVM analog and change how classic vulnerability classes apply:

- Panic-reverts. Contract functions panic on failure (`panic_with_error!`); a panic reverts the transaction. There is no silent-failure or unchecked-return-value class in the ERC20-boolean sense; instead the concern shifts to unintended panics (a single bad element aborting a batch) and to arithmetic traps.
- `overflow-checks = true` in dev and release. Bare `i128` arithmetic traps on overflow rather than wrapping - this converts most silent-overflow bugs into liveness (revert) rather than value-corruption bugs, and makes unguarded divisors and domain-limited `exp` / `ln` inputs the sharper concern.
- Per-transaction CPU + memory metering. A fill that re-values the Blend pool several times can approach the network memory cap (documented as `41,943,040` bytes in the LOE `MEMORY_LIMITS.md`); resource feasibility is a settlement-integrity and denial-of-service concern, not just performance.
- Host-level reentrancy prohibition. A contract already on the call stack cannot be re-entered, which refutes the classic single-contract reentrancy class without an explicit guard. Cross-contract ordering and stale-cache concerns remain.
- Storage tiers and TTL/archival. Instance/Persistent/Temporary tiers with TTL windows and Protocol-23 auto-restoration; split-key state that is not extended can archive, and restoration semantics matter for rate/yield keys.
- SEP-41 / SAC trustlines. Classic-asset transfers require a trustline on the receiver; a payout (LOE maker payout, bridge unlock) to a trustline-less address reverts.

• Nested authorization. Moving a contract's own funds two frames below the contract requires `authorize_as_current_contract` (wrapped here as `authorize_pull`); the completeness of these pre-declarations is a dedicated invariant.

2. Expected invariants

The following properties are the ones a reviewer must hold the code against. The first group is breakable (their violation is a finding); the second is design-enforced (structural, assumed as context).

Breakable invariants (verify):

1. Paired supply - pre-maturity, PT total supply equals YT total supply (mint and burn are always paired). Post-maturity, YT is not burned on redemption.

2. Wrapper solvency - the wrapper's Blend `bToken` position is at least its share supply (`position ≥ supply`). Insolvency is the violation of this and is what the wrapper fuzz tests assert.

3. Round-trip no profit - tokenize→redeem returns strictly ≤ deposited (Floor rounding on both legs), across decimal configurations; no free value extraction from the conversion path.

4. Yield conservation - claimed yield + claimable yield + fees ≤ realized backing gain, held across users, transfers, loss, and recovery.

5. Engine baseline (fill isolation of balances) - a fill preserves the engine's pre-fill IBT/PT/YT balances and settles only its own deltas. The current implementation settles against *live* balances with no entry snapshot, so this invariant is the one to stress.

6. Rate-freeze finality - the maturity rate freeze is one-way, uses the intended epoch, and cannot double-pay.

7. Fill isolation - one invalid or terminal order must not brick unrelated valid orders in the same batch (a panic-reverts consequence given batch atomicity).

8. Router provenance - a successful batch creates no positive Router residual unless explicitly claimable; slippage guards apply to transaction-local deltas, not absolute balances (the Router keeps no per-user accounting, so stranded balances are claimable by anyone).

9. Nested-auth completeness - every nested token pull (the contract's own funds moved a frame below) is correctly pre-declared. The funding-side conversion in LOE `convert_to_ibt` uses a SEP-41 `approve`/allowance mechanism rather than an `authorize_pull` pre-declaration; whether that alternative is complete and equivalent across the Underlying and VaultShare paths is a direct instance of this invariant.

Design-enforced (context only): exactly one PT⇄one YT per market (Factory deploys the pair atomically); `overflow-checks=true` (traps, no wraparound); host forbids reentrancy; `underlying_decimals ≤ ibt_decimals` (checked at `PT.initialize`); fee maxima ≤ `FEE_DIVISOR` (const-asserted); flash loans not ported (dead `PtFlashLoanFee` plumbing only).

3. Trust boundaries and security assumptions

The protocol uses a fine-grained role model (OpenZeppelin stellar-access) with a two-step top-level admin. Roles are the internal trust boundary; three external systems are the out-of-scope trust boundary.

- Admin (2-step) - Trusted. Grants/revokes all roles; two-step transfer. Does not auto-rollover operational roles on handoff.
- UPGRADE - Trusted. Immediate WASM swap (`update_current_contract_wasm`) on any core contract; no timelock, no two-step.
- PAUSER - Trusted (semi). Pause/unpause PT, YT, LOE, Router, wrappers. Factory and Registry have no pause.
- FEE_SET - Semi-trusted. set_tokenization_fee (≤10%), set_yield_fee (≤50%), fee collector, LOE fee rate, reduce_fee. Applied live at settlement/claim - retroactive to accrued yield.
- REGISTRY - Semi-trusted. Peripheral addresses and deployable WASM hashes (immediate).
- RWD_HARV / RWD_PROX - Semi-trusted. Set the rewards proxy (no interface check) and invoke it dynamically as the PT/wrapper.
- RTR_MGR - Semi-trusted. Router set_registry/set_engine/set_bridge/set_broker - authoritative for what the Router executes against.
- RATE_SET - Semi-trusted. Rate-oracle setter (Soroban-only role).

• FACTORY - Contract-only. Registry add_pt/remove_pt, satisfied by the Factory during permissionless deploy_pt.

The assumption for trusted roles (admin, UPGRADE) is that they are honest and secured (multisig/timelock operationally, though no on-chain timelock is enforced). Semi-trusted roles are assumed to act within their advertised bounds; the review treats a semi-trusted role acting maliciously or carelessly as an in-scope likelihood factor rather than an assumption of safety. Two centralization levers deserve explicit note because their blast radius is broad and immediate: (a) `set_rewards_proxy` + `claim_rewards`, which permits an arbitrary call executed as the PT with no target/selector/argument allowlist and no proxy interface check; and (b) upgrade, an immediate logic swap with no timelock or two-step.

4. Attack vectors considered

• By role / access rights. Privilege-abuse and blast-radius analysis for each semi-trusted role (fee inflation up to the caps applied retroactively to accrued yield; Router peripheral redirection via RTR_MGR; arbitrary call-as-PT via the rewards proxy; WASM-hash swap poisoning every future market deploy). Permissionless entry points - register_order, fill_orders, execute, PT deposit/redeem, deploy_pt, wrapper deposit/redeem/withdraw - were treated as attacker-controlled, including initialize front-running (no constructor protection) and permissionless store_rates_at_expiry.

• By value flow. Fee-application timing and denomination; cross-decimal share scaling on both tokenize and redeem legs; first-depositor / zero-rate initialization; dust rounding below SAFETY_BOUND (100 in underlying decimals) flooring yield to zero; round-trip profit; and the Router's lack of per-user accounting (stranded/residual balances claimable by anyone, and Amount::Balance sweep composition collapsing to zero).

• By dependency surface (external return-value trust). Consumption of the broker bought amount and bridge message id; the hand-written #[contractclient] return-type and argument-count correctness; and the wrapper's reconciliation (or lack of it) of minted bToken against priced shares - deposit reconciles and reverts on BTokenMismatch, but withdraw prices off the cached b_rate with no refresh and no post-submit reconciliation, and redeem refreshes but does not reconcile.

• By economic risk. Rate staleness (the PT serves a cached ibt_rate; the wrapper serves a per-timestamp cached b_rate); mispricing in the order curve (zero-price divisors on the

inverted branches; exp/ln domain limits; fail-safe-rate decimal-scale sensitivity); and insolvency / bad-debt propagation from Blend into wrapper solvency and PT redemption.

• By cross-protocol interaction. Blend bad-debt socialization moving b_rate mid-transaction against a frozen timestamp; bridge delivery failure committing an irreversible local lock/burn; broker output trust; and the memory-metered fill path where repeated Blend re-valuation can exhaust the per-transaction envelope and revert a settlement.

5. Components/flows manually reviewed, invariants checked, behaviors tested

The review manually traced the following flows against the invariants above:

• Tokenize (`deposit/deposit_ibt`) - reviewed for fee timing, decimal scaling, nested-auth on the underlying pull, first-depositor init; exercises paired supply and round-trip no profit.

• Redeem / withdraw (`redeem/redeem_for_ibt/withdraw`) - reviewed for yield settle-before-burn ordering, YT clamp, withdraw stale-rate pricing, CEI; exercises paired supply and yield conservation.

• LOE fill (`fill_orders` → `pull_and_price` → `convert_to_ibt` → `settle`) - reviewed for live-balance settlement, funding-path authorization, per-order isolation, fail-safe floors, zero-price divisors, memory feasibility; exercises engine baseline, fill isolation, nested-auth completeness.

• Wrapper `deposit/redeem/withdraw` - reviewed for `b_rate cache` freshness and reconciliation asymmetry, 1 share = 1 bToken; exercises wrapper solvency.

• Router multicall (execute) - reviewed for `Amount::Balance` composition, external return trust, `AssertMinBalance` (absolute vs delta), residual sweeps, nested-auth pre-declarations; exercises Router provenance and nested-auth completeness.

• Bridge-out - reviewed for underfunded-gas committing lock/burn without delivery, client return ABI; (external) - no on-chain recovery.

• Yield claim (`claim_yield/claim_yield_in_ibt`) - reviewed for retroactive fee-rate change on accrued backlog, dust `SAFETY_BOUND`, fee denomination; exercises yield conservation and rate-freeze finality.

6. Test coverage - gaps and caveats

• Auth-model fidelity. Many tests run under `mock_all_auths`, which does not exercise the enforcing authorization tree. Nested-auth completeness (invariant 9) - the exact area where the LOE funding path uses approve instead of an `authorize_pull` pre-declaration - is best validated under enforcing auth, and coverage there is lighter than under mocked auth.

• External-artifact conformance. The broker and the bridge's read-side client ABI are pinned against the live deployed contracts on a mainnet fork. What remains unverified in-tree is the destination-chain (EVM) delivery leg of a bridge-out.

• Cross-decimal breadth. Round-trip-no-profit and yield conservation depend on `underlying_decimals ≤ ibt_decimals` combinations; the fuzz campaign covers the math primitives, but full cross-decimal protocol-level scenarios (especially low-decimal underlyings) are only partially represented.

• Bad-debt / `b_rate` movement. Fork tests cover rate increase; same ledger `b_rate` movement from Blend bad-debt socialization at a frozen timestamp - the driver of the wrapper's withdrawal reconciliation gap - is harder to reproduce.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	1	1	1
Medium	6	5	5
Low	11	11	8
Informational	14	14	11
Total	**32**	**31**	**25**

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
H-01	Blend b_rate is not guaranteed to be monotonic and can corrupt accounting	High	Fixed
M-01	Fail-safe rate uses input-token decimals for output-token floor	Medium	Fixed
M-02	LOE: missing authorize_pull makes Underlying/VaultShare-funded fills always revert in production	Medium	Fixed
M-03	Reported output of external aggregator is trusted	Medium	Fixed
M-04	Fees are changeable on already-accrued YT yield	Medium	Acknowledged
M-05	Both Router bridge-out commands revert on every successful call due to mis-declared return type	Medium	Fixed
M-06	Bridging a PT with underfunded gas commits a local PT lock or wrapped-PT burn while the cross-chain delivery never happens	Medium	Fixed
L-01	Admin transfer requires manual rollover of operational roles	Low	Acknowledged

L-02	Zero YT price lets IbtForYt orders register but revert during fill	Low	Fixed
L-03	Permissionless deploy_pt poisons registered PT list	Low	Acknowledged
L-04	Rewards-proxy can arbitrarily "call-as-self", granting too much power to the harvester role	Low	Acknowledged
L-05	LimitOrderEngine pulls taker_input_token before validating it, stranding unsupported non-swept tokens	Low	Fixed
L-06	Zero-amount third-party redeem/withdraw lets any caller force another user's yield checkpoint	Low	Fixed
L-07	Router AssertMinBalance can miss under-delivery for pre-funded callers	Low	Fixed
L-08	pricing_data returns a stale stored IBT rate where Solidity recomputes it live	Low	Fixed
L-09	RedeemPtForIbt drops Solidity's pre-maturity YT-balance clamp	Low	Fixed
L-10	Unauthenticated initialize is front-runnable on every contract	Low	Fixed
L-11	Per-user tokenization-fee reductions are keyed to the Router/engine's own address, not the user	Low	Fixed

I-01	Partial fills can settle with zero maker proceeds	Informational	Fixed
I-02	Payout-side tokenisation creates an unused YT approval	Informational	Fixed
I-03	PT deployment salt IBT inclusion should be considered	Informational	Acknowledged
I-04	PT initialization can accept a zero initial IBT rate	Informational	Fixed
I-05	Behalf-of redeem requires an undocumented YT allowance	Informational	Fixed
I-06	Wrapper `max_` function quotes are liquidity-blind	Informational	Fixed
I-07	Blend-backed LOE fillability is route-dependent under current Soroban memory limits	Informational	Fixed
I-08	Router bridge commands should reject negative signed amounts before calling the bridge	Informational	Fixed
I-09	LOE fill interface docs incorrectly require taker approval to the engine	Informational	Fixed
I-10	set_secs_per_ledger uses a raw assert instead of a structured contract error	Informational	Fixed
I-11	Zero-amount YT transfer permissionlessly triggers `before_yt_transfer` on any holder	Informational	Acknowledged

I-12	Registry template and upgrade hashes are insta-changeable	Informational	Acknowledged
I-13	Internal token approvals use inconsistent expiry ledgers (seq+1 vs seq)	Informational	Fixed
I-14	Long-metadata tokens can get blocked out of deployment	Informational	Fixed

High Severity Issues

H-01: Blend b_rate is not guaranteed to be monotonic and can corrupt accounting

Severity: High	Impact: High	Likelihood: Medium
Files: blend-fungible-vault-wrapper/src/storage.rs blend-fungible-vault-wrapper/src/lib.rs spectra-fungible-vault-wrapper/src/lib.rs	Status: Fixed	

Description:

The Blend vault wrapper caches the reserve's b_rate (bToken→underlying rate) in temporary storage, keyed on the ledger timestamp, and reuses it for all share↔asset conversions. The design assumes b_rate is monotonic and therefore invariant within a single ledger timestamp. This holds for Blend's time-based interest accrual, but not for bad-debt socialization: when a loss exceeds the backstop, Blend's default_liabilities reduces the rate through a discrete write (reserve.data.b_rate -= b_rate_loss), separate from accrual, reached permissionlessly on a full bad-debt-auction fill, and visible to same-ledger reads (Reserve::load short-circuits on timestamp == last_time). So b_rate can decrease at a frozen timestamp — the case the cache assumes cannot occur. This is correct Blend behavior; the defect is on the Spectra side.

The wrapper holds one shared bToken position and treats 1 share = 1 bToken (no totalSupply/totalAssets accounting). Of the three flows, deposit re-reads the reserve fresh and reconciles the minted bTokens against the priced shares (BTokenMismatch, lib.rs:145,180-182), and redeem calls refresh_b_rate before pricing (lib.rs:204). withdraw does neither — it prices the burn and its max_withdraw cap off the cached value (lib.rs:248), with no refresh and no post-submit reconciliation. The refresh_b_rate doc comment justifies protecting redeem because it "withdraws funds with no post-submit reconciliation"; that reasoning applies verbatim to withdraw, which was left out.

During a same-ledger socialization, withdraw burns ceil(assets / cached_rate) shares while the pool removes ceil(assets / real_rate) bTokens. With cached_rate > real_rate it burns fewer shares than the bTokens removed, so the position drops below the supply. The wrapper's

solvency invariant is position >= supply and insolvency is exactly its violation: supply × b_rate (share liabilities) then exceeds position × b_rate (backing). Nothing reconciles the two afterward, so the break is permanent.

Example:

- Three holders, 100 shares each, position 300 bTokens.
- b_rate 1.0; a 50% socialization drops it to 0.5 (position == supply still holds
- each holder's fair value is now 50):

Alice withdraws on the stale cached 1.0:

burns ceil(100 / 1.0) = 100 shares

pool removes ceil(100 / 0.5) = 200 bTokens (at the real rate)

-> Alice gets 100 (escapes her haircut); position 100 < supply 200 (insolvent by 50)

Bob (fresh rate) withdraws -> 50 (the rest of the position); David -> 0 (his shares are unbacked)

Fair allocation is 50/50/50; the actual is 100/50/0. The loss is split by exit ordering, not by stake. Exploitation requires, within one ledger timestamp:

- a prior on-chain wrapper call to prime the cache at the pre-loss rate,
- a socialization writing b_rate down,
- a withdraw before the timestamp advances or a deposit/redeem refreshes the cache

No attacker is required during a withdrawal surge; a capitalized actor can also arrange the sequence atomically since auction fills are permissionless. This class of event is real on Blend v2, the YieldBlox exploit created bad debt that depleted the backstop.

Recommendations:

Make withdraw refresh b_rate from the pool before pricing, mirroring redeem.

Customer Response:

Fixed in PR-203.

Fix Review:

Fix confirmed.

Medium Severity Issues

M-01: Fail-safe rate uses input-token decimals for output-token floor

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: spectra-common/src/limit_order_engine.rs:79-84 limit-order-engine/src/lib.rs:898-942 principal-token/src/lib.rs:68-74	Status: Fixed	

Description:

`Order.fail_safe_rate` is documented as an 18-decimal WAD slippage floor for wrapper conversions. The engine applies it as:

```
let floor = mul_div_floor(env, amount, order.fail_safe_rate, pricing::UNIT);
```

and then compares the result against the wrapper conversion output:

```
let out = client.deposit(&amount, &engine, &engine, &engine);
if out < floor {
    panic_with_error!(env, LimitOrderEngineError::FailSafeRateNotMet);
}
```

This strips the WAD scaling correctly but leaves floor in the raw decimal scale of the input amount, while the output it is compared against uses the output token's scale. With a 6-decimal underlying and an 18-decimal IBT:

```
amount        = 100_000_000        // 100 underlying, 6 decimals
fail_safe_rate = 0.99e18           // 1% slippage

floor = amount * fail_safe_rate / 1e18
      = 99_000_000
```

The intended minimum is 99 IBT shares (`99e18` raw); the implemented floor is `99_000_000` raw — 10^{12} times too small, so any output passes.

Since `PrincipalToken::initialize` enforces `underlying_decimals <= ibt_decimals`, the two reachable cases are: funding-side underlying→IBT conversions, where the floor is too small and the check is bypassed; and payout-side IBT→underlying conversions, where the floor is too large and valid fills revert. The same unscaled floor is passed as `min_shares` / `min_vault_shares` on the VaultShare paths.

Existing tests miss this: the cross-decimal matrix sets `fail_safe_rate = 0`, and all nonzero fail-safe tests use the 18/18 setup where the scales coincide.

On mixed-decimal markets the fail-safe floor does not enforce the intended slippage bound: funding-side fills can execute at an arbitrarily degraded conversion rate (the maker's protection is silently ineffective), and payout-side fills with any meaningful nonzero rate always revert, making the order unfillable unless the maker disables the floor.

Recommendations:

Normalize the floor into the output token's raw scale:

```
floor = amount * fail_safe_rate * 10^out_decimals / 1e18 / 10^in_decimals
```

and use the same normalized value for `min_shares` / `min_vault_shares`.

Customer Response:

Fixed in PR-205.

Fix Review:

Fix confirmed.

M-02: LOE: missing authorize_pull makes Underlying/VaultShare-funded fills always revert in production

Severity: Medium	Impact: High	Likelihood: Low
Files: limit-order-engine/src/lib.rs:889-904 limit-order-engine/src/lib.rs:905-916 blend-fungible-vault-wrapper/src/lib.rs:152-157 principal-token/src/lib.rs:380-384 spectra-common/src/auth.rs:33-34	Status: Fixed	

Description:

When a funding-side order (`IbtForPt` / `IbtForYt`) is funded in Underlying or VaultShare, `fill_orders` pulls the maker's leg token into the engine, then `convert_to_ibt` authorizes the wrapper with `approve` and deposits:

```
token::Client::new(env, leg_token).approve(&engine, ibt, &amount, &live_until);
let out = client.deposit(&amount, &engine, &engine, &engine); // (assets, receiver, from, operator)
```

With `from == operator == engine`, the Blend wrapper pulls the asset via `t.transfer(&from, &wrapper, &assets)`, which requires `engine.require_auth()` one frame below the engine's own call (engine → wrapper.deposit → asset.transfer). A contract's automatic authorization only covers its direct invocations; a deeper call made on its behalf must be pre-declared with `env.authorize_as_current_contract` (wrapped here as `authorize_pull`). The engine never does this, and the `approve` it issues is dead: `transfer` — unlike `transfer_from` — never reads allowances. Under enforcing authorization, the transfer has no backing entry and the host reverts.

`PrincipalToken` performs the identical operation correctly — same wrapper, same `operator == from` branch, same depth — calling `authorize_pull` first (`principal-token/src/lib.rs:380-384`), with a comment naming the exact trap.

The suite misses this because every wrapper-backed test runs under `mock_all_auths_allowing_non_root_auth()`, adopted to accommodate the *legitimate* sub-auths the wrapper and PT make — plain `mock_all_auths()` rejects those. This can be

reproduced in the tests by swapping `mock_all_auths_allowing_non_root_auth()` for plain `mock_all_auths()` in any Underlying-funded fill test and the fill reverts with an authorization error. See `multi_denom_fill::repro_*`: the same fill passes/reverts on exactly that mock swap, while an Ibt-funded fill passes under both.

The soroban sdk documentation warn about this exact scenario: link

Every Underlying-funded funding-side order registers cleanly but reverts deterministically when filled — the mode is unusable in production; only Ibt funding works. No funds are lost: fills revert atomically and the maker keeps custody. Since `fill_orders` is atomic, one such order reverts an entire batch. The auth mock makes the break invisible pre-deployment.

Recommendations:

In both `convert_to_ibt` branches, authorize the nested pull the way `PrincipalToken` does, before the `deposit` / `wrap_with_min_shares` call:

```
spectra_common::auth::authorize_pull(env, leg_token, &engine, ibt, amount);
```

Customer Response:

Fixed in PR-206.

Fix Review:

Fix confirmed.

M-03: Reported output of external aggregator is trusted

Severity: Medium	Impact: High	Likelihood: Low
Files: router/src/dispatcher.rs:248-295 router/src/dispatcher.rs:67-70/:305-309 spectra-common/src/router_types.rs:72-75 src/lib.rs:122	Status: Fixed	

Description:

The `BrokerSwap` command hands a swap to **StellarBroker**, a third-party DEX aggregator the router calls as an external contract — live on mainnet (`CBWP275BNGLHWFTQVB6QHA67MLX7WTX6YZ5LNSUVOK7W2TMKWX7OPYOJ`, routing across Aqua/Soroswap/Comet/Phoenix), present in this repo only as a client interface and a governance-set address. The router trusts that integration further than it should: after authorizing the broker to pull `token_in` and calling `swap`, it enforces slippage purely against the value the broker **reports back**, never against tokens the router actually receives.

```
let bought = result.get(1)...;            // the broker's self-reported output
if bought < args.min_amount_out { panic!(AssertMinBalanceFailed); }
```

`BrokerSwapArgs` carries no output token and the router takes no balance delta, so `min_amount_out` validates the aggregator's word rather than reality. The one safeguard that *does* check reality — a `Command::AssertMinBalance` on the router's actual `token_out` balance — is **optional**: the caller has to remember to append it. So the protective check is opt-in while the trusting path is the default, which is backwards for an external dependency.

Selecting a trusted aggregator is not the same as trusting it to answer honestly on every call. A bug, an upgrade, a compromise, or an output-token quirk in StellarBroker all make the reported `bought` diverge from what the router holds — none of which require compromising Spectra's own roles. And when it diverges the loss is silent, because the standard route delivers with a balance sweep (`Amount::Balance`, fork_test.rs:139-154) and transferring `0` does not revert: the broker

takes `token_in`, returns a satisfying `bought` it never delivered, the check passes, the user receives `0`, and `execute` succeeds with the input gone.

Recommendations:

Make output verification obligatory inside the swap instead of leaving it to the caller. In `dispatch_broker_swap`, take `token_out` (add it to `BrokerSwapArgs`, or derive it from the route and require every route to end in the same token), snapshot the router's `token_out` balance before `swap`, and assert `balance_after - balance_before >= min_amount_out` afterward — verifying receipt rather than trusting the report, so `min_amount_out` means what callers assume.

Customer Response:

Fixed in PR-213.

Fix Review:

Fix confirmed.

M-04: Fees are changeable on already-accrued YT yield

Severity: Medium	Impact: Low	Likelihood: High
Files: principal-token/src/lib.rs:872-885, 1190-1204	Status: Acknowledged	

Description:

The yield fee is applied at claim time using the fee rate *currently* set, not the rate(s) in effect while the yield accrued. `update_yield_inner` accumulates accrued yield into the stored running backlog `data.yield_in_ibt` gross - the fee is not applied here. `_claim_yield` zeroes the backlog and computes `compute_yield_fee(env, yield_in_ibt)` on the full gross backlog. `compute_yield_fee` reads the registry rate live on every call - `RegistryClient::new(env, &registry_addr).get_yield_fee()`), then `mul_div_wide(gross_yield_in_ibt, yield_fee_rate, FEE_DIVISOR, Ceil)`.

No code path snapshots the fee rate at accrual time or settles the backlog when `set_yield_fee` changes the rate, so a rate increase applies retroactively to the entire outstanding backlog.

Recommendations:

Mitigation is complex and requires a re-design of the entire fee system. If the fee is never intended to change after deployment, it can be considered to acknowledge and accept the risk. "subsumes under-quote" overstates

Customer Response:

Acknowledged.

M-05: Both Router bridge-out commands revert on every successful call due to mis-declared return type

Severity: Medium	Impact: Medium	Likelihood: High
Files: contracts/router/src/dispatcher.rs:18-44 contracts/router/src/dispatcher.rs:183-192 contracts/router/src/dispatcher.rs:236-245	Status: Fixed	

Description:

When the Router bridges a PT to another chain it calls the external `PtBridge` contract's `bridge_native_pt_to_evm` / `bridge_back_to_evm`. To make those calls the Router uses a hand-written interface that declares both as returning nothing:

```
// contracts/router/src/dispatcher.rs:18-44 — the Router's HAND-WRITTEN bridge client
#[contractclient(name = "BridgeClient")]
pub trait Bridge {
    fn bridge_native_pt_to_evm(
        env: Env, sender: Address, pt: Address, amount: i128, target_chain_id: u32,
        evm_recipient: BytesN<20>, gas_amount: i128,
        refund_address: Option<Address>, min_amount_out: i128,
    );                                      // ← no -> ...  ⇒ result decoded as ()
    fn bridge_back_to_evm(/* same 8 args */);   // ← same: no return type
}
```

However both bridge entry points return a 32-byte message ID on success.

On Soroban, a client that declared a `()` return traps when the callee hands back a non-void value, and the trap reverts the whole transaction. So every successful bridge-out call reverts and both Router bridge commands are unusable in production. Atomic rollback means no funds are lost or stranded.

Recommendations:

Declare the two bridge client methods as returning `BytesN<32>`. Better, generate the client from the deployed bridge spec instead of hand-writing it, plus a CI compatibility test against the pinned `PtBridge` WASM.

Customer Response:

Fixed in PR-211.

Fix Review:

Fix confirmed.

M-06: Bridging a PT with underfunded gas commits a local PT lock or wrapped-PT burn while the cross-chain delivery never happens

Severity: Medium	Impact: High	Likelihood: Low
Files: contracts/router/src/dispatcher.rs:166-193 contracts/router/src/dispatcher.rs::207-246 contracts/router/src/dispatcher.rs:305-310	Status: Fixed	

Description:

Bridging your PT to Ethereum requires attaching gas to pay the relayer, but the Router lets you attach zero and if you do, your PT is still locked/burned on Stellar while the cross-chain message is created but never paid for, so it's never delivered and no on-chain call on either side can give your PT back. E.g Alice bridges 100 PT with gas_amount = 0 or her frontend under-quoted -> her 100 PT is locked in the Stellar bridge, the transaction "succeeds," but the message sits unpaid forever -> Alice never receives her 100 PT on Ethereum and the locked PT is unrecoverable.

Bridging a PT to EVM does two things in one Soroban transaction:

1. The native PT is pulled into the bridge and recorded as locked, or the wrapped PT is burned

2. An outbound message (Axelar) telling the destination chain to mint/release. The Router lets the caller pass zero gas and only wires up a gas payment when gas is positive. In the real bridge the local lock/burn is committed before the messenger call, and the messenger, given zero gas, skips paying the gas service but still emits the gateway call and returns success. There is no error and no Soroban rollback: the PT is durably locked/burned, the bridge returns a message ID as if it succeeded, message sits unfunded in the called state, never relayed.

The only recovery is off-chain - paying for the Axelar gas out-of-band.

Note: The project's `docs/E2E-BRIDGE-GUIDE.md` records the exact outcome: a Stellar→EVM bridge with `gas_amount 0` "stays stuck at called.", however this is a case that could and should be handled on-chain via enforcement, thus the write-up and severity assignment.

Recommendations:

Add `RouterError::InsufficientGas` and a non-zero, admin-configurable `min_gas_floor`, ideally keyed by `target_chain_id` / `gas_token` checked before the bridge call. Subsumes zero, under-quote, and negative gas.

Customer Response:

Fixed in PR-230.

Fix Review:

Fix confirmed.

Low Severity Issues

L-01: Admin transfer requires manual rollover of operational roles

Severity: Low	Impact: Low	Likelihood: Low
Files: principal-token/src/lib.rs:92-97/883-888 spectra-fungible-vault-wrapper/src/lib.rs:313-356 contracts/blend-fungible-vault-wrapper/src/lib.rs:55-59 docs/security/ownable2step-design.md:13	Status: Acknowledged	

Description:

The two-step admin transfer flow rotates only the top-level `stellar_access` admin slot. It does not automatically revoke or migrate operational roles that were granted to the original admin during initialization.

As a result, after a successful admin handoff, the previous admin can continue using any operational roles they already held until the new admin explicitly revokes them. Although this is consistent with the stated design, it creates a real operational risk: a handoff can appear complete while one or more privileged roles remain assigned to the outgoing admin.

Recommendations:

Precautions should be made to ensure that any admin hand-off also results in a full roll-over of all other roles. A stale admin should not be able to have any effect on the protocol.

Customer Response:

Acknowledged.

L-02: Zero YT price lets IbtForYt orders register but revert during fill

Severity: Low	Impact: Low	Likelihood: Low
Files: limit-order-engine/src/lib.rs:773-804 & 549-562 limit-order-engine/src/pricing.rs:66-79 & 142-150	Status: Fixed	

Description:

`register_order` rejects negative `implied_rate` values, but it accepts `implied_rate == 0`. For PT pricing, a zero implied rate means there is no time discount, so the PT price equals the future PT value. The corresponding YT price is calculated as:

```
yt_price = future_pt_value - pt_price
```

At `implied_rate == 0`, this evaluates to zero.

This creates an edge case for `IbtForYt` orders. That branch prices the order by inverting the YT price:

```
let spot_yt = yt_price_in_underlying(env, implied_rate, time_to_maturity,
future_pt_value);
let spot = mul_div_unit(env, UNIT, ibt_rate_unit, spot_yt);
```

When `spot_yt == 0`, the fill path divides by zero and reverts. The order can therefore be registered successfully, but cannot be filled.

The same issue can occur if a very small positive implied rate, short time to maturity, or fixed-point rounding causes `yt_price_in_underlying` to round down to zero.

A maker can register an apparently valid `IbtForYt` order (registration takes no escrow) that reverts when priced. `fill_orders` prices orders in one uncaught loop, so including this order in a batch reverts the *entire* batch rather than skipping the bad order — the solver or taker has to resubmit without it. No funds are at risk; the impact is batch griefing, recoverable by excluding the order.

Recommendations:

Reject `implied_rate == 0` in `validate_order_for_registration`. This closes the practical trigger. Note the underlying fault is a zero *divisor*: a dust-sized positive rate can also floor the YT price to zero, so the airtight fix is to guard the computed divisor in the inverted-price branches (`IbtForYt`, `IbtForPt`) — panic on a zero divisor instead of dividing.

Customer Response:

Fixed in PR-219.

Fix Review:

Fix confirmed.

L-03: Permissionless `deploy_pt` poisons registered PT list

Severity: Low	Impact: Low	Likelihood: Medium
Files: contracts/factory/src/lib.rs:86-123 contracts/registry/src/lib.rs:246-253	Status: Acknowledged	

Description:

`deploy_pt` is not role-gated. Its entire authorization is a self-auth and a non-zero duration. Only the registry's `add_pt` is `#[only_role(caller, "FACTORY")]`:

```
// contracts/registry/src/lib.rs:246-253
#[only_role(caller, "FACTORY")]
pub fn add_pt(env: &Env, caller: Address, pt: Address) {
    if storage::is_registered_pt(env, &pt) {
        panic_with_error!(env, RegistryError::PtListUpdateFailed);
    }
    storage::set_pt_registered(env, &pt);
    PtAdded { pt }.publish(env);
}
```

However the `[only_role("FACTORY")]` check is satisfied because `add_pt` is called as `reg_client.add_pt(&env.current_contract_address(), &pt)` during the deployment of the PT. The unauthenticated end-user supplies any `IBT` and rides the Factory's role through to an unconditional `set_pt_registered`. The role gate provides zero merit - it stops a random address from calling `add_pt` directly, but it does nothing to stop anyone from reaching `add_pt` via `deploy_pt` with any backing asset.

Also, `PrincipalToken::initialize` only probes the supplied IBT's interface, it does not validate that the IBT is honest or real. Thus we can fill the list with random tokens.

Recommendation:

Consumers must be aware and treat the registry's PT list as untrusted and resolve through a curated allowlist. The Solidity original is permissionless here too, so this is a documentation/integration contract rather than a code change.

If on-chain divergence from Solidity is wanted: add a DEPLOYER role

Customer Response:

Acknowledged.

L-04: Rewards-proxy can arbitrarily "call-as-self", granting too much power to the harvester role

Severity: Low	Impact: High	Likelihood: Low
Files: contracts/principal-token/src/lib.rs:640-664	Status: Acknowledged	

Description:

`claim_rewards` is Spectra's Soroban stand-in for Solidity's `delegatecall` style rewards-harvest pattern. It performs a fully dynamic cross-contract call into a stored proxy address, with both the function selector and the argument vector supplied by the caller, without containing any sanitation of the addresses, no target allowlist, no selector allowlist, and no argument checks:

```
// contracts/principal-token/src/lib.rs:604-629
#[only_role(caller, "RWD_PROX")]
fn set_rewards_proxy(env: &Env, caller: Address, proxy: Option<Address>) {
    let old = storage::try_get_rewards_proxy(env);
    match &proxy {
        Some(addr) => storage::set_rewards_proxy(env, addr),  // any Address; no
code-exists/interface check
        None => storage::remove_rewards_proxy(env),
    }
    crate::events::RewardsProxyChange { old, new: proxy }.publish(env);
}

#[only_role(caller, "RWD_HARV")]
#[when_not_paused]
fn claim_rewards(env: &Env, caller: Address, function: Symbol, args: Vec<Val>) ->
Val {
    let proxy = match storage::try_get_rewards_proxy(env) {
        Some(p) => p,
        None => panic_with_error!(env, PrincipalTokenError::NoRewardsProxy),
    };
```

```
    let result = env.invoke_contract::<Val>(&proxy, &function, args); // PT is
the DIRECT invoker
    crate::events::RewardsClaimed { caller }.publish(env);
    result
}
```

Thus the principal token is the direct invoker of `env.invoke_contract` and is granted the direct invoker auto-authorization for the duration of that call

This converts the rewards-harvest entry point into an arbitrary call as PT whose blast radius is everything a PT is authorized to do:

1. Drain the IBT backing. With `set_rewards_proxy(Some(IBT))` then `claim_rewards("transfer", [PT, attacker, backing])`, the IBT token records `from = PT` (auto-auth) and transfers the PT's entire IBT position to the attacker. Every PT holder's redemption is now unbacked.
2. Mint unbacked YT / break the paired-supply invariant. With `proxy = YT` and `claim_rewards("mint", [attacker, shares])`, PT auto-satisfies `YieldToken::mint`'s `pt.require_auth()` and mints YT with no matching PT, violating breakable invariant #1 (`PT supply == YT supply`, recon §13).
3. Replace the YT code. `claim_rewards("upgrade", [evil_yt_wasm])` reaches `YieldToken::upgrade` (L128-131) — a cross-role escalation, since legitimate YT code-rolls are reserved for the `UPGRADE`-gated `upgrade_yt` (L589).

Recommendations:

Two-layer guard at all sites in the shared trait so the Blend wrapper inherits it:

1. `set_rewards_proxy` rejects sensitive targets (self, underlying/IBT token, YT, vault-share, Blend pool)
2. `claim_rewards` selector denylist for asset-moving/authority selectors (`transfer`, `transfer_from`, `burn`, `mint`, `upgrade`, `submit`, …).

Customer Response:

Acknowledged.

L-05: LimitOrderEngine pulls taker_input_token before validating it, stranding unsupported non-swept tokens

Severity: Low	Impact: Low	Likelihood: Low
Files: limit-order-engine/src/lib.rs:246-252 limit-order-engine/src/lib.rs:649-679 limit-order-engine/src/lib.rs:597-601 limit-order-engine/src/lib.rs:721-731	Status: Fixed	

Description:

`fill_orders` transfers `taker_input_amount` of the caller-supplied `taker_input_token` into the engine *before* any check that it is a token the engine can use:

```
soroban_sdk::token::Client::new(env, &taker_input_token).transfer(&caller,
&engine, &taker_input_amount);
```

`handle_tokenisation` only converts the supported inputs — `ibt` (for `IbtForPt`/`IbtForYt`) and `pt`/`yt` (for `YtForIbt`/`PtForIbt`). Any other token falls into the `else` branch, which silently assumes the caller paid the canonical `taker_token` and does nothing. Settlement then verifies only that the engine holds enough of the *canonical* token (`transfer_post_validation`), and `settle_taker` sweeps back only `ibt`/`pt`/`yt`. An unsupported `taker_input_token` outside that swept token set is therefore never converted and never swept.

The canonical-balance check usually turns this into a harmless revert: each fill sweeps the engine's full `ibt`/`pt`/`yt` balance to the receiver, so the engine should normally hold no canonical balance between transactions. A wrong token then leaves the engine short of the canonical token, and the whole transaction reverts atomically, refunding the caller. The token only strands when the engine already holds a canonical balance ≥ `total_taking`, which requires a direct donation or other pre-existing canonical residue before the next fill sweeps it. In that case the fill settles against that existing canonical balance, the caller receives their output, and the unsupported non-swept token is left in the engine with no recovery path.

The defect is the ordering: the engine accepts and pulls a token before verifying it is one it supports. Settlement amounts remain internally consistent, but the funding source is wrong: the current fill can use old canonical balance while the token supplied by the caller is ignored and stranded.

Recommendations:

Validate that `taker_input_token` is one of the inputs supported for the order type — the canonical `taker_token`, or the convertible `ibt`/`pt`/`yt` — and reject otherwise, *before* the `transfer`. Prefer up-front rejection over having `settle_taker` also sweep the residual: sweeping would still let a wrong-token fill settle against (and drain) any pre-existing canonical balance, whereas rejecting first prevents the pull entirely.

Customer Response:

Fixed in PR-220.

Fix Review:

Fix confirmed.

L-06: Zero-amount third-party redeem/withdraw lets any caller force another user's yield checkpoint

Severity: Low	Impact: Low	Likelihood: High
Files: principal-token/src/lib.rs:447-452 principal-token/src/lib.rs:535-542 principal-token/src/lib.rs:1381-1390 yield-token/src/lib.rs:214-224 principal-token/src/lib.rs:277	Status: Fixed	

Description:

Several PT and YT entry points settle a user's yield *before* they enforce the authorization that would normally protect that user, and that authorization is either a burn or a SEP-41 allowance spend. Both are trivially satisfied at `amount == 0`: a zero burn and a zero `spend_allowance` pass against a missing (zero) allowance, and only the *caller's* own signature is required. So any account can drive these paths against any `owner` and force that owner's yield settlement without the owner's consent.

Two entry points expose this:

- `redeem` / `redeem_for_ibt` / `withdraw` / `withdraw_ibt` **(PrincipalToken).** These omit a top-level `caller.require_auth()` and rely on the burn for authorization. `_before_redeem` calls `update_yield_inner(owner, ...)` — advancing the owner's `UserYield` checkpoint — before the burns. For a zero amount both burns reduce to `Base::burn_from(caller, owner, 0)`, which passes on the caller's own signature and a trivially-satisfied `spend_allowance(owner, caller, 0)`. So `redeem(attacker, 0, attacker, victim)` mutates the victim's checkpoint with nothing but the attacker's signature.

- `transfer_from` **(YieldToken).** It calls `before_yt_transfer(from, to, ...)`, which settles both sides' yield and is gated only on the caller being the paired YT, before `Base::transfer_from`. For a zero amount the transfer needs no allowance and only the spender's auth, so `yt.transfer_from(attacker, victim, attacker, 0)` settles the victim's yield on the attacker's signature alone. The plain `transfer` path also settles first, but

`Base::transfer` requires `from.require_auth()`, so a third party cannot drive it; only `transfer_from` is exposed.

The core problem is isolation, not magnitude: **one account should never be able to reach into and mutate another account's yield-accounting state without that account's authorization.** Here an unauthorized third party can checkpoint any holder, repeatedly and at will, through either path. That this is reachable at all is the defect.

The concrete consequence is minor, which is what keeps the severity Low: each forced checkpoint drops the sub-unit yield remainder (`compute_yield` stores `yield_in_ibt` floored via `from_ray`), so repeated forcing shaves yield dust off the victim, proportionally more for a small holder forced every ledger. It costs the attacker a transaction per checkpoint, yields them nothing, and puts no principal or claimable yield at risk. No fees are triggered either, since fees are charged on claim, not during settlement.

The YT `transfer_from` ordering is inherited from the Solidity reference (`YieldToken.transferFrom` also calls `beforeYtTransfer` before `super.transferFrom`), so that vector is a shared design property rather than a port regression. It is still reachable in the deployed Soroban contract and the fix is cheap, so it is worth closing alongside the PT paths.

Impact

Any account can advance any other account's yield checkpoint, with no allowance or authorization from the victim, through a zero-amount redeem/withdraw or a zero-amount YT `transfer_from`. The victim loses yield dust to forced rounding, more of a small balance's yield under sustained forcing. The griefing is unprofitable and bounded, and steals no principal or yield, but it is an unauthorized effect on another user's state, which should not be possible regardless of how small the loss is.

Recommendations:

Short-circuit zero-amount operations before any yield settlement, the same way `YieldToken::burn` already does (`if amount == 0 { return }`):

- In `redeem` / `redeem_for_ibt` (`shares == 0`), `withdraw` (`assets == 0`), and `withdraw_ibt` (`ibts == 0`): return early before `_before_redeem` / `_before_withdraw` reaches `update_yield_inner`.

- In `YieldToken::transfer` and `transfer_from` (`amount == 0`): return before calling `before_yt_transfer`.

This also removes the pointless zero burns, zero transfers, and their events. Requiring the owner's authorization for a zero-amount third-party call would also close it, but the short-circuit is simpler, matches the existing `burn` guard, and matches the expectation that a no-op does nothing. Add a regression test where an attacker with no allowance calls both `redeem(victim, 0, ...)` and `yt.transfer_from(victim, attacker, 0)` and asserts the victim's `UserYield` / `UserYieldData` is unchanged.

Customer Response:

Fixed in PR-221.

Fix Review:

Fix confirmed.

L-07: Router AssertMinBalance can miss under-delivery for pre-funded callers

Severity: Low	Impact: Medium	Likelihood: Low
Files: router/src/dispatcher.rs:72-80 router/src/dispatcher.rs:82-101	Status: Fixed	

Description:

`AssertMinBalance` resolves `owner` (`Caller`, `Router`, or any address) and reverts when that owner's **total** token balance is below `min_value`:

```
Command::AssertMinBalance(args) => {
    if args.min_value < 0 {
        panic_with_error!(env, RouterError::NegativeMinValue);
    }
    let owner = resolve_recipient(caller, &router, &args.owner);
    let bal = token::Client::new(env, &args.token).balance(&owner);
    if bal < args.min_value {
        panic_with_error!(env, RouterError::AssertMinBalanceFailed);
    }
}
```

It answers "does this wallet hold at least `min_value` in total?", not "did this batch deliver at least `min_value`?". Those are equivalent only when the checked wallet started with no output-token balance.

The safe pattern is to route output to the Router, assert `owner=Router`, then `Transfer(Balance -> Caller)`. The Router normally starts with no user-owned output balance, so the absolute check acts like a delta check.

The unsafe direct-delivery pattern pays the caller directly, for example `DepositAssetInIbt { recipient: Caller }`, then checks `AssertMinBalance(owner=Caller)`. If the caller already holds enough output token, the assert can pass even if the batch under-delivers. Example: a caller holding 1,000 IBT expects about 500 more shares and sets `min_value = 495`; if the route returns only 300, the final 1,300 balance still passes.

This matters most for `DepositAssetInIbt` and `RedeemIbtForAsset`, which have no native min-out fields. By contrast, the PT legs use `min_shares` / `min_ibts`. Solidity has the same absolute `ASSERT_MIN_BALANCE` check, so this is an inherited design rather than a Soroban port regression.

A caller or integration using direct-to-caller delivery with `AssertMinBalance(owner=Caller)` can lose its intended slippage protection whenever the caller already holds the output token. A sandwiched or mispriced route can commit instead of reverting. No third party gains fund access; the loss is bound to the caller's own bad batch.

Recommendations:

Keep the contract semantics for Solidity parity; close the gap in documentation and routing helpers:

- Document in `contracts/router/README.md` that `AssertMinBalance` is an absolute-balance check, that `owner=Caller` is not a slippage guard on pre-funded wallets, and that the supported pattern for legs without native min-out is `recipient=Router → AssertMinBalance(owner=Router) → Transfer(Balance → Caller)`.
- Ensure any first-party SDK/route-builder emits the Router-sweep pattern rather than direct delivery when a balance assert is the only guard.
- Optionally (deliberate divergence from upstream), add a delta-based assert — snapshot the owner's balance when the command vector begins executing and compare the increase — which would make direct-to-caller delivery safe as well.

Customer Response:

Fixed in PR-225.

Fix Review:

Fix confirmed.

L-08: pricing_data returns a stale stored IBT rate where Solidity recomputes it live

Severity: Low	Impact: Low	Likelihood: Medium
Files: principal-token/src/lib.rs:184-192 principal-token/src/lib.rs:327-329 principal-token/src/lib.rs:1089-1121 principal-token/src/lib.rs:1134-1152 limit-order-engine/src/pricing.rs:100-102 & 977-985 & 953-968	Status: Fixed	

Description:

`pricing_data` returns `(ibt_rate, future_pt_value, expiry)`. Element 0 is read straight from storage:

```
pub fn pricing_data(env: &Env) -> (i128, i128, u64) {
    let ibt_rate = storage::get_ibt_rate(env);                    // cached checkpoint value
    let preview = <Self as PrincipalToken>::preview_redeem(env, ONE); // resolves rates live
    ...
    (ibt_rate, future_pt_value, storage::get_expiry(env))
}
```

The stored `IbtRate` is initialized at deployment, frozen at expiry, and normally refreshed pre-expiry by `_update_pt_and_ibt_rates`, which runs on deposit / redeem / withdraw / yield settlement / YT transfer. Between those refreshes, it lags the live rate (`preview_redeem(ibt_unit)` in `_get_current_pt_and_ibt_rates`) by whatever yield the IBT has accrued since the last such call.

The limit-order-engine's `get_price` uses element 0 as the `ibt_rate_unit` divisor that converts underlying-denominated PT/YT prices into IBT terms in all pricing branches. When it is stale, the IBT-denominated leg is off by a factor of `live_ibt_rate / stored_ibt_rate`.

This is a Solidity parity divergence: `getIBTRate()` recomputes the rate live on every call via `_getCurrentPTandIBTRates` (a fresh `previewRedeem(ibtUnit)` read), whereas the Soroban getter - and `pricing_data` - return the cached value. The staleness is a pre-expiry effect only; post-expiry both sides return the frozen rate.

Limit-order fills priced off a PT whose stored IBT rate has drifted from live vault reality mispriced by the accrued-drift factor when no rate-refreshing PT/YT operation occurs before `get_price`. Some paths refresh first: YT `transfer` / `transfer_from` calls `before_yt_transfer`, which refreshes PT/IBT rates before pricing. Other common paths, such as PT-side fills and IBT-funded fills whose tokenization runs after pricing, can still use the stale tuple value.

The effect is usually small and bounded by market idleness - actively-used PTs stay near-live because ordinary PT/YT activity refreshes the stored rate. It can be larger for an idle PT or an IBT rate jump. The result is a value transfer between fill counterparties, not a protocol drain.

Recommendations:

Have `pricing_data` return the IBT rate from the same live resolution used for `future_pt_value`, while preserving the current paused behavior. `preview_redeem(ONE)` already flows through `_get_current_pt_and_ibt_rates → ibt_client.preview_redeem(ibt_unit)`, so the live rate is computed on this call path and then discarded in favor of the stored one. Derive both tuple elements from one live `_get_pt_and_ibt_rates` resolution instead of reading `storage::get_ibt_rate`; if that bypasses `preview_redeem`, add an explicit pause guard so paused PTs still fail closed.

Customer Response:

Fixed in PR-223.

Fix Review:

Fix confirmed.

L-09: `RedeemPtForIbt` drops Solidity's pre-maturity YT-balance clamp

Severity: Low	Impact: Low	Likelihood: Medium
Files: contracts/router/src/dispatcher.rs:115-122 contracts/principal-token/src/lib.rs:1301-1313 contracts/principal-token/src/lib.rs:1371-1374	Status: Fixed	

Description:

Solidity's `Dispatcher._dispatch` clamps a pre-maturity PT redeem to the router's own YT balance: `redeemShares = block.timestamp < maturity ? min(shares, YT.balanceOf(router)) : shares`. The Soroban `Command::RedeemPtForIbt` forwards the resolved `shares` or `Amount::Balance`with no clamp which pre-maturity reverts at `_before_redeem`'s `shares > _max_burnable(owner)` guard so if the router's PT balance exceeds its YT balance the whole `multicall` aborts with `InsufficientBalance`. Reachable when the user sold/moved part of their YT, the deposit leg routed YT elsewhere, or a prior command spent YT, no fund loss.

Recommendations:

Clamp `shares = min(shares, YT.balance(router))` pre-maturity in the dispatcher arm (both `maturity()` and `get_yt()` are on `PrincipalTokenClient`)

Customer Response:

Fixed in PR-222.

Fix Review:

Fix confirmed.

L-10: Unauthenticated initialize is front-runnable on every contract

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/registry/src/lib.rs:21-34 contracts/factory/src/lib.rs:23-38 limit-order-engine/src/lib.rs:62-101 blend-fungible-vault-wrapper/src/lib.rs:53-72 scripts/core:530-533 scripts/core:545-550	Status: Fixed	

Description: Currently there is no `__constructor` on any initializer - only an `is_initialized` sentinel. Non-atomic `safe_deploy` then separate initialization lets the first caller win admin + all roles during the normal deployment flow via front-running. While it is more difficult on Stella, core contracts are vulnerable to this front-running during initial deployment. If we look at `scripts/core`, the deployment happens in two separate CLI commands:

```
# Transaction 1: Deploy
local factory; factory=$(safe_deploy "$factory_wasm")

# Transaction 2: Initialize
safe_invoke "$factory" initialize --admin "$admin" --registry "$registry"
```

Recommendations:

Core contracts that are deployed via CLI bash scripts, should migrate the initialize logic into a `__constructor` function. Modern versions of the Stellar CLI also allow you to pass constructor arguments directly during the deploy step.

Customer Response:

Fixed in PR-227.

Fix Review

Fix confirmed.

L-11: Per-user tokenization-fee reductions are keyed to the Router/engine's own address, not the user

Severity: Low	Impact: Low	Likelihood: Medium
Files: principal-token/src/lib.rs:1253-1261 router/src/dispatcher.rs:113 limit-order-engine/src/lib.rs:668	Status: Fixed	

Description:

The discount is keyed on the direct `caller`, but on every Router-mediated or LOE-filled deposit the caller is the shared Router/engine contract, so a granted per-user reduction is looked up against the wrong principal. Two directions: a discounted user routing through the Router silently pays the full fee, thus their discount dropped OR a reduction mis-granted (by `FEE_SET`) to the Router address subsidizes every routed deposit.

This is a fee-level mischarge, not a theft of principal or of another user's funds. The overcharge is largely self-avoidable - a partner who only wants to deposit calls `PT.deposit_ibt` directly and gets the exact discount, so it evaporates only on composed deposits forced through the Router. The LOE leg has no entitled victim and the amplified "Router subsidizes every routed deposit" direction needs a `FEE_SET` admin misconfiguration.

Issue #50 (PT README:165) is preview-only -the bare 4626 preview understates the discount — and it explicitly assumes the actual deposit delivers the discount This shows the actual deposit does not deliver it through the Router/LOE, which #50 does not cover and arguably contradicts.

Recommendations:

Add a user-aware deposit path so the Router/LOE can pass the authenticated economic user into `get_fee_reduction`, or document that per-user reductions do not apply to routed/filled deposits and remove the feature's implied coverage there.

Customer Response:

Fixed in PR-224.

Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Partial fills can settle with zero maker proceeds

Files:

limit-order-engine/src/lib.rs

Description:

`fill_orders` allows the taker to choose a partial `making_amount` for each order. For every positive partial fill, the engine reduces the order's remaining amount, pulls the maker's token, and computes maker proceeds with floor rounding:

```
let actual_taking = mul_div_floor(env, spot_price, ibt_for_pricing,
pricing::UNIT);
```

There is no guard that `actual_taking` is nonzero when `actual_making` is positive. Therefore a positive fill can settle with zero maker payment whenever:

```
spot_price * actual_making < 1e18
```

In that case, the maker's filled amount is updated, the maker token is pulled into the engine, and settlement transfers `0` to the maker. The remaining engine balance is then swept to the taker's receiver.

This is most visible for very small partial fills or YT orders close to maturity, where the YT price approaches zero. It is not a material drain by itself: each zero-payment fill can move less than one raw unit of the payment token's value. Since PT initialization rejects token decimals below 6, this is bounded to less than `0.000001` payment tokens per fill for supported PT/YT/IBT markets.

Recommendations:

Add a positive-proceeds guard after pricing:

```
if net_making > 0 && actual_taking == 0 {
    panic_with_error!(env, LimitOrderEngineError::ZeroTaking);
```

```
}
```

This hardens the fill path and makes the invariant explicit without relying on off-chain minimum-fill filtering.

Customer Response:

Fixed in PR-215.

Fix Review:

Fix confirmed.

I-02: Payout-side tokenisation creates an unused YT approval

Files:

limit-order-engine/src/lib.rs:641-675
principal-token/src/lib.rs:1371-1388
yield-token/src/lib.rs:86-94

Description:

n the payout-side tokenisation branch, the limit order engine approves the PT contract to spend the engine's YT balance before calling `redeem_for_ibt`:

```
soroban_sdk::token::Client::new(env, yt).approve(
    &engine,
    &pt,
    &redeem_amount,
    &live_until,
);
pt_client.redeem_for_ibt(&engine, &redeem_amount, &engine, &engine);
```

This approval is not consumed by the subsequent redemption. `PT::_before_redeem` calls:

```
YieldTokenClient::new(env, &yt_addr).burn_without_yield_update(owner, caller,
&shares);
```

For this LOE path, both `owner` and `caller` are the engine, so YT receives:

```
owner = engine
spender = engine
```

`YT::burn_without_yield_update` only uses allowance in the `owner != spender` branch. Since `owner == spender`, it burns directly from the engine's YT balance and never reads the `engine -> PT` allowance.

This does not break redemption or authorization. YT requires `pt.require_auth()`, and PT is the direct caller of the YT burn. The engine authorization is needed one frame earlier for the PT-side

burn, where the engine directly invoked PT. No `authorize_as_current_contract` entry is required for this path.

Impact

No user funds are at risk and the redemption path is not blocked. The approval is dead code that can confuse reviewers and future maintainers into thinking the YT burn depends on allowance.

The approval also expires at the current ledger sequence, so it does not create a persistent dangling allowance.

Recommendations:

Remove the unused YT approval from the payout-side tokenisation branch, or update the surrounding comments to clarify that `redeem_for_ibt` burns YT through the `owner == spender` path and does not consume allowance.

Customer Response:

Fixed in PR-215.

Fix Review:

Fix confirmed.

I-03: PT deployment salt IBT inclusion should be considered

Files:

factory/src/lib.rs:125-130
factory/src/lib.rs:86-110

Description:

The Factory derives each PT deployment salt from `sha256(duration || counter)`:

```
fn compute_salt(env: &Env, duration: u64, counter: u64) -> BytesN<32> {
    let mut buf = Bytes::new(env);
    buf.extend_from_array(&duration.to_be_bytes());
    buf.extend_from_array(&counter.to_be_bytes());
    env.crypto().sha256(&buf).into()
}
```

The salt commits to the maturity duration and Factory sequence number, but not to the IBT. Since a PT market is economically defined by `(IBT, duration)`, including the IBT in the salt is a useful hardening improvement: it makes counterfactual PT addresses collateral-bound, so an address computed for one IBT cannot correspond to another IBT at the same duration and counter. This reduces integration mistakes in frontends, indexers, deployment scripts, and off-chain order systems that may reason about deterministic addresses before the deployment event is indexed.

Recommendations:

Include the IBT address in the salt, for example `sha256(duration || counter || ibt)`. If the protocol wants one canonical PT per market, use `sha256(ibt || duration)` and reject duplicate `(IBT, duration)` deployments instead.

Customer Response:

Acknowledged.

I-04: PT initialization can accept a zero initial IBT rate

Files:

PrincipalToken/lib.rs:55
Factory/lib.rs:08-110

Description:

`initialize` only rejects an IBT when `total_assets()` returns zero. It then asks the IBT how much underlying one full IBT unit redeems for and stores that value as the initial IBT rate.

```
if ibt_client.total_assets() == 0 {
    panic_with_error!(env, PrincipalTokenError::RateError);
}

let ibt_rate_raw = ibt_client.preview_redeem(&ibt_unit);
let ibt_rate = ray_math::to_ray(ibt_rate_raw, underlying_decimals);
```

A positive asset total does not guarantee that one full IBT unit is worth at least one underlying base unit. If the vault has very small assets relative to its share supply, `preview_redeem(ibt_unit)` can round down to zero. `to_ray(0, decimals)` keeps that value as zero, so initialization can complete with `ibt_rate` set to zero.

When the PT is deployed through the Factory, the Factory registers it right after initialization. That means a PT with a zero initial IBT rate can become a registered market even though the main flows do not handle that state usefully. Redemption style conversions reject a zero IBT rate with `RateError`, and deposit style flows can compute zero PT shares and revert through the zero share guard.

This does not create direct fund loss because failed Soroban calls revert atomically. It is still a poor market state to admit at initialization because the registry can contain a market that is unusable until the IBT rate becomes positive again.

Recommendations:

After computing `ibt_rate_raw`, reject `ibt_rate_raw <= 0` before storing the rate. The check should cover both zero and negative values because the value is an `i128`, while rates are meant to be unsigned economic quantities.

Customer Response:

Fixed in PR-217.

Fix Review:

Fix confirmed.

I-05: Behalf-of redeem requires an undocumented YT allowance

Files:

contracts/yield-token/src/lib.rs
contracts/principal-token/src/lib.rs
contracts/principal-token/README

Description:

For a pre-expiry behalf-of redeem, PT burns the paired YT via `burn_without_yield_update`, which on `owner != spender` spends a YT allowance in addition to the documented PT allowance. An integrator who grants only the README-documented PT allowance has this YT `burn_from` panic `InsufficientAllowance` before the PT burn is reached.

Recommendations:

Correct the documentation - state in the redeem-flow table that a pre-expiry behalf-of redeem also requires a YT allowance

Customer Response:

Fixed in PR-217.

Fix Review:
Fix confirmed.

I-06: Wrapper `max_` function quotes are liquidity-blind

Files:

contracts/blend-fungible-vault-wrapper/src/lib.rs

Description:

The Blend adapter doesn't override the abstract base's `max_*`, which report the full position with no venue-liquidity awareness. The actual defect is that the `max_*` views are liquidity-blind, ERC-4626 inaccurate. That defect fails safe - atomic rollback, no fund loss, read-only view, no exploit. However, an integrator/front-end that trusts the quote attempts a redeem that reverts.

Recommendations:

Override all three `max_*` functions in the Blend wrapper, reading the reserve once:

- `max_deposit` → 0 if paused/disabled else supply-cap headroom in underlying
- `max_withdraw` / `max_redeem` → 0 if paused, else clamp to the pool's serviceable underlying balance. Confirm `supply_cap` denomination against the Blend source first.

Customer Response:

Fixed in PR-229.

Fix Review:

Fix confirmed.

I-07: Blend-backed LOE fillability is route-dependent under current Soroban memory limits

Files:

limit-order-engine/src/pricing.rs:90-102
principal-token/src/lib.rs:184-190

Description:

The protocol already identified and documented that some Blend-backed LimitOrderEngine routes exceed the current Soroban per-transaction memory cap. We include this only as an audit trace of a known issue, because unsupported Blend-backed routes should not reach production as normal fillable paths.

Recommendations:

Adopt the protocol's proposed engine-level fix: make pricing take the IBT-to-asset rate as caller-supplied input instead of reading it cross-contract during fill_orders. This keeps the Blend pool out of the LOE invocation and removes the dominant memory cost from the fill path.
To avoid regressions or decoupling, consider alternatively to:

1. cache the NAV valuation once per fill batch and thread it through `deposit_ibt`/`redeem_for_ibt`/`YT-transfer` (`*_with(rates)` overload), collapsing 3–5 Blend reads to 1
2. Gate/allowlist supported Blend routes at order registration so unsupported heavy routes cannot be registered as fillable.

Customer Response:

Fixed in PR-231.

Fix Review:

Fix confirmed.

I-08: Router bridge commands should reject negative signed amounts before calling the bridge

Files:

router/src/dispatcher.rs:166-192
router/src/dispatcher.rs:207-245
spectra-common/src/router_types.rs:146-168

Description:

The bridge Router commands use signed `i128` fields for values that are economically unsigned. `resolve_amount` returns `Amount::Exact(x)` as provided, and the bridge dispatchers pass the resolved values to the external bridge without first checking that they are non-negative.

```
let lock_amount = resolve_amount(env, &args.pt, router, &args.amount);
let gas_amount = resolve_amount(env, &args.gas_token, router, &args.gas_amount);

bridge_client.bridge_native_pt_to_evm(
    router,
    &args.pt,
    &lock_amount,
    &args.target_chain_id,
    &args.evm_recipient,
    &gas_amount,
    &Some(refund),
    &args.min_amount_out,
);
```

The same pattern exists in the bridge-back path for `burn_total`, `gas_amount`, and `min_amount_out`.

This is mainly a boundary hardening issue. Standard SEP-41 token transfers should reject negative token amounts, so a negative bridge amount will often revert once the bridge tries to pull or burn tokens. However, the Router still permits malformed signed values such as negative `gas_amount` or negative `min_amount_out` to reach an external cross-chain component. Other Router paths already reject comparable negative slippage values before dispatch, for example `BrokerSwap` rejects negative `min_amount_out`.

Recommendations:

Reject negative bridge values in the Router before constructing auth entries or calling the bridge, checking `lock_amount`, `burn_total`, `gas_amount`, and `min_amount_out` explicitly, and use checked arithmetic for the bridge-back fee calculation. Additionally reject `args.min_shares < 0` and `args.min_ibts < 0` in `Command::DepositIbtInPt` and `Command::RedeemPtForIbt`, which share the same gap.

This is extending an existing pattern rather than adding new validation. `BrokerSwap` already rejects a negative `min_amount_out` with `RouterError::NegativeMinAmountOut` (`router/src/dispatcher.rs:252`). Reuse the same guard and the same error in `dispatch_bridge_native` and `dispatch_bridge_back`, so the bridge commands are consistent with the broker command instead of relying on the external bridge to reject malformed amounts.

Customer Response:

Fixed in PR-218.

Fix Review:

Fix confirmed.

I-09: LOE fill interface docs incorrectly require taker approval to the engine

Files:

spectra-common/src/limit_order_engine.rs:146-148
limit-order-engine/src/lib.rs:246-251
router/src/dispatcher.rs:134-141

Description:

The shared LimitOrderEngine interface documentation says the taker must hold and have approved `taker_input_amount` of `taker_input_token` to the engine before calling `fill_orders`.

```
/// Fill a batch of orders. caller is the taker. must hold and
/// have approved taker_input_amount of taker_input_token to the engine
/// before this call.
```

The implementation does not consume a taker allowance to the engine. For a direct LOE fill, `fill_orders` relies on `caller.require_auth()` and then calls SEP-41 `transfer` from the caller to the engine:

```
if taker_input_amount > 0 {
    soroban_sdk::token::Client::new(env, &taker_input_token).transfer(
        &caller,
        &engine,
        &taker_input_amount,
    );
}
```

When the Router is used, the Router authorizes its own pull into the LOE path with `authorize_pull` before calling `fill_orders`. That still does not require or consume a taker allowance granted to the LOE itself.

This is an interface documentation issue, not a runtime vulnerability. Integrators following the generated client or trait documentation may build an unnecessary approval flow, or misunderstand which contract is expected to hold authorization for routed fills.

Recommendations:

Update the interface documentation to describe the actual authorization model. Direct LOE callers must authorize the `caller` address used in `fill_orders`, and routed fills are authorized by the Router before it invokes the LOE. Do not state that the taker must approve the engine unless the implementation is changed to consume such an allowance.

Customer Response:

Fixed in PR-214.

Fix Review:

Fix confirmed.

I-10: set_secs_per_ledger uses a raw assert instead of a structured contract error

Files:

spectra-common/src/ttl.rs:37-40

Description:

`set_secs_per_ledger` validates caller-provided configuration with a raw `assert!`:

```
pub fn set_secs_per_ledger(env: &Env, secs: u64) {
    assert!(secs > 0, "secs_per_ledger must be positive");
    env.storage().instance().set(&SECS_PER_LEDGER_KEY, &secs);
}
```

Project guidance requires reachable contract input failures to use structured `#[contracterror]` variants via `panic_with_error!`. Raw `assert!` is appropriate for internal invariants that should not be reachable through normal input. Here, `secs` is an admin/configuration input, so `0` is a reachable invalid value.

Impact

This does not create a protocol-security issue. It reduces error consistency, makes client handling less precise, and leaves this helper inconsistent with the rest of the workspace's structured failure style.

Recommendations:

Add a structured error variant for an invalid seconds-per-ledger value and replace the raw `assert!` with `panic_with_error!`. Update the existing zero-value regression test to expect the structured contract error rather than a string panic.

Customer Response:

Fixed in PR-214.

Fix Review:

Fix confirmed.

I-11: Zero-amount YT transfer permissionlessly triggers `before_yt_transfer` on any holder

Files:

yield-token/src/lib.rs:199-225
principal-token/src/lib.rs:901-904

Description:

`transfer` only guards `if is_expired(e) && amount != 0 { panic }` - for `amount == 0` it is a no-op regardless of expiry. It then reads `from_bal`/`to_bal` and calls `before_yt_transfer(from, to, from_bal, to_bal)` *before* `Base::transfer` - `to.require_auth()` is never called on either path. `transfer_from` has the same structure. `before_yt_transfer` in the PT gates only on `get_yt(env).require_auth()`, then runs `update_yield_inner` for both `from` and `to` - a state-mutating yield settlement on the address with no authorization, permissionlessly mutating state for no cost.

Recommendation:

Mirror the shipped `burn` guard - add `if amount == 0 { return; }` right after the expiry check in both `transfer` and `transfer_from`, before any balance read or `before_yt_transfer`.

Customer Response:

Acknowledged.

I-12: Registry template and upgrade hashes are insta-changeable

Files:

registry/src/lib.rs:125-145
registry/src/lib.rs:292-300

Description:

`set_pt_wasm_hash` / `set_yt_wasm_hash` are guarded only by `#[only_role(caller, "REGISTRY")]` and immediately persist the new hash - no timelock, no two-step, the REGISTRY role alone suffices. Similarly, upgrade is guarded by `#[only_role(caller, "UPGRADE")]` and calls `env.deployer().update_current_contract_wasm(new_wasm_hash)` directly, an immediate single-call bytecode replacement with no delay or multi-party gate. The Factory deploys new PT/YT instances from these registry-held hashes, so a changed hash propagates to every subsequently created pair.

This is a privileged-role design property rather than a logic bug and the same no-timelock pattern covers `set_factory` / `set_router`/ `set_fee_collector` / `set_bridge` / `set_limit_order_engine`, the fee setters, and every `upgrade` path

Recommendations:

One shared propose→execute-after-delay timelock applied uniformly to all privileged mutators + upgrade paths, with a bounded emergency-guardian escape for upgrades; range-bounded fee setters may stay immediate.

Customer Response:

Acknowledged.

I-13: Internal token approvals use inconsistent expiry ledgers (seq+1 vs seq)

Files:

router/src/dispatcher.rs:111
limit-order-engine/src/lib.rs:646
limit-order-engine/src/lib.rs:891,907
router/README.md:22

Description:

Both the Router and the limit-order-engine implement the same one-shot pattern: grant a short-lived internal SEP-41 allowance immediately before a `transfer_from`-based call that should consume it in the same transaction. They set different expiry ledgers — the Router uses `sequence() + 1`, while the engine uses `sequence()` for PT tokenization and wrapper conversion approvals.

A SEP-41 allowance is valid while `live_until_ledger >= current_sequence`, so both work and both are usable within the current transaction. `sequence()` is therefore sufficient for same-invocation internal approvals. The Router's `sequence() + 1` window stays valid one extra ledger if the approved callee does not consume the full amount. In the intended PT flow the allowance is consumed immediately, but there is no functional reason for the two internal patterns to differ.

Impact

No functional impact for honest callees that consume the approved amount in-transaction. The inconsistency is primarily a maintenance/readability wart. The tighter `sequence()` pattern also avoids a one-ledger residual allowance window if a buggy or untrusted callee under-consumes.

Recommendations:

Standardize internal one-shot approvals on `sequence()` where the allowance is consumed in the same invocation, and update the Router README. If the Router intentionally keeps `sequence() + 1`, document why the extra ledger is needed.

Customer Response:

Fixed in PR-218.

Fix Review:

Fix confirmed.

I-14: Long-metadata tokens can get blocked out of deployment

Files:

spectra-common/naming.rs

Description:

`naming.rs` copies an external token's symbol/name into fixed 32/128/192-byte stack buffers with no length guard. A token symbol ≥33 B (wrapper name ≥162 B / symbol ≥62 B) makes PT/YT/wrapper `initialize` revert on an unstructured slice-panic → that yield source is permanently unsupportable. Solidity's `NamingUtil.sol` uses dynamic strings, abi.encodePacked, arbitrary length, but the Soroban rewrite introduced the fixed buffers, so this is a slight port regression - behavior that worked in the original now reverts.

Recommendations:

Length-check with a structured error instead of the default panicking. Size the intermediate for the full 128 B output where possible.

Customer Response:

Fixed in PR-214.

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.